Candidate-Level Boundary Matching for Tokenized Autonomous Driving

Alba Maria Tellez Fernandez
amtellezfernandez@gmail.com
Independent research prototype

Abstract: Candidate-based autonomous-driving planners fail for different reasons that are often collapsed into one aggregate score. Sometimes no generated candidate is safe. Sometimes a safe candidate exists but the planner selects an unsafe top-1 action. Sometimes the top-1 action is safe but leaves substantial progress or imitation value on the table. We call these three cases a generation gap, recoverable safety regret, and recoverable utility regret. This paper introduces REGRETBOOT, a model-agnostic audit and bootstrapping pipeline that turns those distinctions into training targets for candidate generators. The method scores every candidate under public log-replay geometry, decomposes each scene by oracle@K versus top-1 value, emits candidate-expansion requests when no in-bank safe teacher exists, and trains a candidate-level replay-value student on recovered oracle@K targets. On 1,000 interaction-aware public nuPlan scenes, a nine-token ManeuverToken bank has 281 generation gaps, 346 recoverable safety regrets, and a 0.627 top-1 replay-failure rate. Expanding the bank to 21 candidates reduces oracle@K replay failure from 0.281 to 0.223, showing that richer candidate support directly shrinks the generation gap. On one illustrative DB-level holdout split, a scene-token bootstrap student underuses this richer bank and still fails at rate 0.460, while a candidate-level replay-value student reaches 0.097 replay failure, matching that split's oracle boundary, and a balanced value objective retains 14.278 m mean progress versus the oracle's 11.258 m. Across five repeated DB-level splits, safety-heavy replay value averages 0.218 ± 0.177 replay failure versus the oracle's 0.208 ± 0.175, while a balanced point averages 0.289 ± 0.164 with higher progress at 19.512 ± 4.513 m. The central finding is therefore structural: candidate expansion addresses the generation branch, but the effective learning unit for recoverable regret is candidate-level boundary matching, not scene-level token classification. The current evidence is deliberately bounded: it is public nuPlan log replay with a ManeuverToken surrogate generator, not proof of closed-loop VLA safety. The contribution is the recoverable-regret training loop, the generation-versus-selection decomposition, and the candidate-level replay-value student that approximates the local oracle boundary inside each candidate bank.

Keywords: autonomous driving, candidate generation, imitation learning, nuPlan, ManeuverToken, replay safety, recoverable regret, bootstrapping

1 Introduction

Modern autonomous-driving policies increasingly generate multiple possible futures, score them, and execute one candidate. This candidate interface appears in hand-designed token planners, imitation-learned trajectory heads, and vision-language-action (VLA) planners. It creates an important diagnostic opportunity: when the executed action fails, we can ask whether the planner failed to *generate* a good action or merely failed to *select* one. Standard aggregate metrics hide that distinction.

10th Conference on Robot Learning (CoRL 2026), Austin TX, USA.

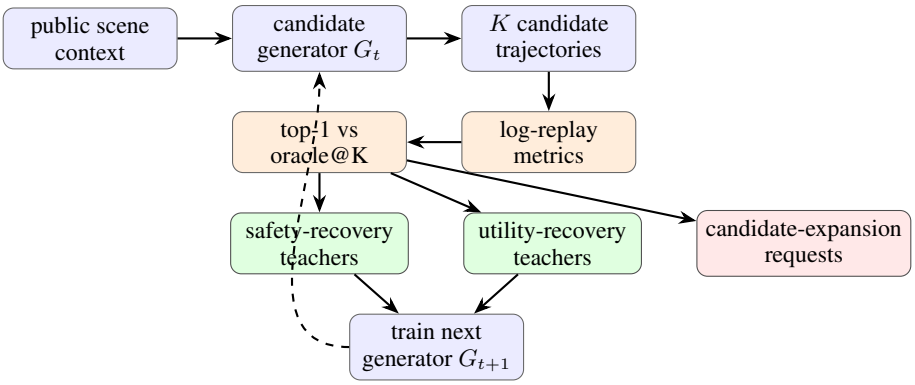

Figure 1: REGRETBOOT separates candidate-generation failure from recoverable selection regret. Safety and utility cases produce in-bank teacher trajectories for bootstrapping. Generation-gap cases do not: they request a richer candidate distribution.

This paper argues that the useful unit of analysis is recoverable regret. Given a scene s, a candidate set $\mathcal{C}(s) = \{\tau_i\}_{i=1}^{K}$, and the planner's top-1 candidate τ_1, we compare top-1 to an oracle over the same candidate bank. If oracle@K also fails, the generator has a candidate-generation gap. If oracle@K is safe but top-1 fails, the failure is recoverable safety regret: the candidate bank already contained a safer trajectory. If both are safe but oracle@K has higher value, the failure is recoverable utility regret: the planner can preserve more progress or expert similarity without changing the generator class. These cases require different fixes, so they should not be optimized with one undifferentiated reranking loss.

REGRETBOOT uses this decomposition as a bootstrapping loop. It does not merely rerank a fixed bank at inference. It produces stage-specific supervision for the next generator iteration: safety-recovery targets where the generator already proposed a safe alternative, utility-recovery targets once safety regret is mostly removed, and candidate-expansion requests when no safe in-bank teacher exists. The intended VLA use case is direct: a VLA planner proposes K trajectory candidates, the replay-value teacher selects high-value in-bank targets, and the generator is distilled toward those targets. The present paper validates the loop on a controlled ManeuverToken surrogate before claiming a large VLA improvement. The main empirical correction in this draft is that the student must operate at the candidate level. A scene-only token classifier can reduce some replay failure, but it leaves a large gap to oracle@K once the candidate bank becomes richer. A candidate-level replay-value student can exploit that richer bank directly.

The empirical result is intentionally narrow and reproducible. We use public nuPlan scenes [1], an interaction-aware sampler, and logged future actors to evaluate the replay clearance of every token. The current generator is a ManeuverToken bank of nine canonical responses. This surrogate is useful because every candidate can be scored exactly and the top-1 choice can be changed without confounding the candidate set. The same JSON/JSONL candidate interface also accepts external VLA outputs, but VLA dumps are used for regret claims only after every candidate has replay and utility metrics attached.

We make four contributions.

1. **Recoverable-regret decomposition.** We define generation gap, recoverable safety regret, and recoverable utility regret for candidate driving planners using top-1 versus oracle@K under the same candidate bank.
2. **Bootstrapped candidate curriculum.** We turn the decomposition into a generator-training curriculum: safety recovery first, utility recovery second, and candidate expansion for scenes with no safe in-bank teacher.

3. **Public interaction replay audit.** On 1,000 public nuPlan interaction scenes, the initial token generator exposes all three regimes at scale: 281 generation gaps, 346 recoverable safety regrets, and 44 utility regrets.
4. **Generation versus boundary-matching evidence.** Expanding the candidate bank from 9 to 21 ManeuverTokens reduces oracle@K replay failure from 0.281 to 0.223. On an illustrative holdout split, a scene-token student still fails at 0.460, while a candidate-level replay-value student reaches the same 0.097 replay-failure boundary as the oracle. Across five repeated DB-level splits, safety-heavy replay value remains within a 0.05 oracle-gap tolerance in 5/5 splits, while the balanced point remains within that tolerance in 3/5 splits and improves progress over the safety-heavy point in 5/5 splits.

2 Related Work

Imitation learning and long-tail driving. Behavior cloning suffers from covariate shift [2, 3], and driving systems often degrade on rare interaction compositions even when common-case performance is strong [4, 5]. Larger sensor-fusion and VLA models improve representation and scale [6, 7], but they do not by themselves answer whether a failure is due to missing candidates or bad selection among candidates.

Candidate planners and VLA grounding. Latent VLA driving systems such as LaST-VLA [8] argue that planning should be grounded in geometry and dynamics rather than only textual reasoning. Our work is complementary: it instruments the action-candidate interface that such systems must ultimately use. A grounded generator can still fail if its top-1 candidate is not the best candidate it generated. Conversely, no reranker can recover a scene where oracle@K is unsafe. The paper's core distinction is therefore between generation gap and recoverable selection regret.

Reranking, self-training, and bootstrapping. Candidate reranking is a known way to improve top-1 decisions. REGRETBOOT is not presented as a novel scoring head alone. Its contribution is to use replay-measured regret to decide which examples should train the next generator iteration and which examples require candidate expansion, and to show empirically that the right learning unit is candidate-level replay value rather than scene-level token classification. The safety-recovery and utility-recovery phases produce teacher trajectories; generation-gap scenes do not. This prevents the method from treating all planner failures as more labels for the same selector.

3 Method

3.1 Candidate Interface

For each scene s, a planner or generator G_t emits a finite candidate bank

$$\mathcal{C}_t(s) = \{\tau_1, \ldots, \tau_K\},$$

where each τ_i is a trajectory candidate with an identifier, optional token label, and per-candidate metrics. The current nuPlan surrogate uses the nine ManeuverTokens in Table 1; the same file format accepts sampled or beam-search trajectories from a VLA generator.

Each candidate is evaluated against logged future actors. A candidate is replay-failed if its minimum replay clearance falls below the near-miss threshold. This is a conservative future-geometry test, not a reactive closed-loop simulator: logged actors do not respond to the ego candidate.

3.2 Replay Value

We assign each candidate a replay value

$$V(s,\tau) = -\lambda_f \mathbf{1}[\tau \text{ replay-fails}] + w_p\, p(s,\tau) - w_a\, \mathrm{ADE}_{3s}(s,\tau) - w_f\, \mathrm{FDE}_{3s}(s,\tau) + w_c\, c(s,\tau), \quad (1)$$

Table 1: ManeuverToken candidate bank used for the public surrogate experiment. The method does not depend on these hand-written tokens; it only requires a candidate bank with attached replay and utility metrics.

Token	Speed scale	Lateral offset	Role
STOP	0.00	0.0	hard stop
CRAWL	0.15	0.0	cautious forward
MAINTAIN	1.00	0.0	nominal progress
SLOW_YIELD	0.50	0.0	yielding progress
NUDGE_LEFT	0.80	+0.9	mild left avoidance
NUDGE_RIGHT	0.80	-0.9	mild right avoidance
EVASIVE_LEFT	0.70	+2.2	strong left avoidance
EVASIVE_RIGHT	0.70	-2.2	strong right avoidance
LANE_RECOVER	0.90	0.0	lane-center recovery

where p is route progress, ADE_{3s} and FDE_{3s} compare the candidate to logged ego motion, and c is optional clearance reward. The main reported artifacts use $\lambda_f = 250$, $w_p = 1$, $w_a = 2$, $w_f = 0$, and $w_c = 0$. These weights are not claimed to be universal; they define the replay teacher used for the current audit.

Let

$$\tau^\star(s) = \arg\max_{\tau \in \mathcal{C}_t(s)} V(s, \tau)$$

be oracle@K inside the generated bank, and let $\tau_1(s)$ be the generator's selected top-1 candidate. The recoverable regret is

$$R(s) = \max(0, V(s, \tau^\star) - V(s, \tau_1)). \tag{2}$$

3.3 Boundary-Matching Student

The learned object in the current draft is not a generic reranker over scene labels. It is a candidate-level boundary matcher. For each scene–candidate pair (s, τ), the student receives the same per-candidate replay and utility features used by the audit and predicts

$$\hat{V}_\phi(s, \tau) \approx V(s, \tau).$$

Selection is then

$$\hat{\tau}_\phi(s) = \arg\max_{\tau \in \mathcal{C}_t(s)} \hat{V}_\phi(s, \tau).$$

This matters because the decision boundary is defined inside each candidate bank. The student is successful only if it reproduces the oracle ordering that separates replay-failed candidates from replay-safe candidates with maximal value. A scene-token classifier collapses this structure into one scene label and therefore cannot express within-bank tradeoffs once the bank becomes richer. The candidate-level replay-value student instead learns the local oracle boundary directly.

3.4 Regret Taxonomy

Each valid scene is assigned one of four outcomes:

Generation gap. Oracle@K replay-fails. No candidate in the bank is safe under the replay metric, so there is no in-bank teacher trajectory.

Recoverable safety regret. Oracle@K is replay-safe but top-1 replay-fails. The candidate bank contains a safe teacher that the generator failed to select.

Recoverable utility regret. Oracle@K and top-1 are both replay-safe, but $V(s, \tau^\star) - V(s, \tau_1)$ exceeds a utility threshold. The scene is safe but inefficient or poorly matched to logged ego motion.

Minor regret. Top-1 is replay-safe and the value gap is below the utility threshold.

The taxonomy is deliberately ordered. Safety regret takes precedence over utility regret; generation gap takes precedence over both because no in-bank target can repair the scene.

3.5 Bootstrapped Curriculum

REGRETBOOT converts the taxonomy into training data for the next generator iteration. For each recoverable safety-regret scene, the target is the oracle@K candidate and the loss weight is proportional to normalized regret. In the candidate-level student used in this paper, the same scenes also provide dense pairwise supervision across the entire bank: unsafe low-value candidates should score below the oracle candidate, and safe but suboptimal candidates should be ordered by replay value rather than collapsed into one token class. After a safety-recovery pass, the pipeline re-audits the new generator G_1 and extracts utility-recovery targets from the remaining safe-but-suboptimal scenes. Generation-gap scenes are emitted as candidate-expansion requests instead of training labels, because distilling toward an unsafe oracle would teach the generator to imitate a failure.

This staging is the key methodological point. A single reranker can only exploit existing candidate diversity. REGRETBOOT asks whether the generator itself improved: recoverable safety regret should fall after safety bootstrapping; utility regret may rise because safe but low-utility decisions become visible; generation gap should remain unchanged unless the candidate distribution expands. The candidate-level student therefore has one concrete job: approximate the oracle boundary on the current bank well enough that the next generator iteration can be distilled toward the right in-bank targets.

4 Experiments

4.1 Public Interaction Replay Setup

We use a 1,000-scene interaction-aware public nuPlan replay subset. Scenes are sampled to avoid the smallest/easiest databases and to prefer dynamic interactions using actor density, minimum actor distance, closing/TTC-style risk when available, ego speed, route/intersection context, and expert braking/steering magnitude. Each scene has nine ManeuverToken candidates and replay metrics for all candidates, producing 9,000 scored candidate trajectories. The validator marks the dump as analysis-ready: all 1,000 scenes and all 9,000 candidates have replay and utility metrics attached.

Claim boundary. The reported replay metric uses logged future actor geometry. It supports claims about future-geometry infeasibility and recoverable candidate regret. It does not prove closed-loop controller safety, because actors do not react to ego actions. We use the closed-loop bridge only as infrastructure in this draft, not as the main evidence claim.

4.2 Initial Regret Decomposition

Table 2 shows the main lifecycle result. The initial nine-token generator G_0 has 281 generation gaps, 346 recoverable safety regrets, and 44 recoverable utility regrets. Its top-1 replay-failure rate is 0.627, while oracle@K replay-failure rate is 0.281. The difference is the recoverable selection surface: many failures are not caused by the absence of any safe candidate, but by selecting the wrong candidate from a bank that already contains a safer one.

4.3 Expanded Candidate Bank

Expanding the bank from 9 to 21 ManeuverTokens changes the generation boundary directly. On the same 1,000-scene public interaction replay study, oracle@K replay failure drops from 0.281 to 0.223, so the generation gap falls from 281 to 223 scenes. This is the first empirical point the paper needs: candidate expansion is not cosmetic; it creates new safe in-bank teachers. However, the proxy top-1 heuristic on the expanded bank worsens to 0.676 replay failure, because a richer bank without a stronger student simply exposes more selection mistakes.

Table 2: Recoverable-regret lifecycle on 1,000 interaction-aware public nuPlan scenes. The candidate-level replay-value student closes most recoverable safety regret on one illustrative split of a fixed bank. Candidate expansion changes the oracle boundary; student choice determines how much of that boundary is actually exploited.

Stage	Gen. gap	Safety regret	Utility regret	Mean regret	Top-1 fail	Oracle fail
G_0 9-token bank	281	346	44	79.649	0.627	0.281
expanded 21-token oracle@K	223	453	22	103.238	0.676	0.223
expanded + scene-token student	223	n/a	n/a	85.707	0.460	0.097
expanded + replay-value student	223	n/a	n/a	21.146	0.097	0.097

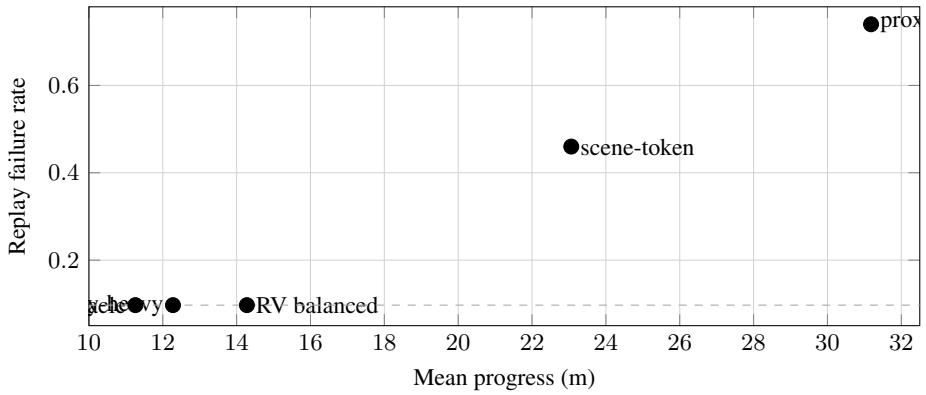

Figure 2: Illustrative holdout safety–utility border on the expanded 21-token bank. The scene-token student remains far above the oracle failure boundary. Candidate-level replay value (RV) reaches the same 0.097 failure line as the oracle on this split, and the balanced RV point retains more progress than the safety-heavy point while staying on that boundary.

4.4 Student Choice Matters

We compared two students on the expanded bank. The first is a scene-token classifier that predicts one token label from scene features. The second is a candidate-level replay-value MLP that scores each candidate directly under the replay objective. The distinction is decisive. On one illustrative DB-level split, the scene-token student reduces holdout replay failure from 0.740 to 0.460, but still leaves a large gap to the expanded-bank oracle at 0.097. In contrast, the candidate-level replay-value student reaches the oracle failure boundary exactly on that split: both fail at 0.097.

This is the desired methodological separation. Candidate expansion changes what is recoverable by oracle@K. Student design determines how much of that recoverable surface is actually exploited. A scene-only token classifier underuses the richer bank. A candidate-level value student can exploit it.

Figure 2 makes the same result visible as a boundary comparison rather than an isolated table lookup. The important point is not that replay value is "better" in the abstract. It is that candidate-level replay value moves the selector onto the expanded-bank failure boundary, while the balanced objective dominates the more conservative point on progress without giving up failure rate.

4.5 Boundary Consistency Across DB Splits

The illustrative split above is useful for intuition, but it is not the whole claim. We therefore reran the same comparison across five DB-level splits of the expanded 1,000-scene replay surface. Table 4 shows the aggregated result. The scene-token student does not provide a robust method signal: it averages 0.590 ± 0.110 replay failure versus the proxy's 0.602 ± 0.130 and improves over proxy in only 3/5 splits. Candidate-level replay value is materially different. The safety-heavy point averages 0.218 ± 0.177 replay failure against the oracle's 0.208 ± 0.175 and stays within a 0.05 oracle-gap tolerance on 5/5 splits. The balanced point averages 0.289 ± 0.164 replay failure with higher progress

Table 3: Illustrative holdout comparison on the expanded 21-token bank. The scene-token student leaves a large gap to the oracle boundary. Candidate-level replay value reaches the oracle failure boundary on this split, and a balanced replay-value objective preserves more progress than the more conservative value point.

Selector	Replay fail	Progress (m)	ADE3 (m)	Entropy
proxy top-1	0.740	31.181	2.714	2.773
scene-token student	0.460	23.062	4.737	2.471
replay-value, safety-heavy	0.097	12.276	9.190	3.089
replay-value, balanced	0.097	14.278	8.451	3.117
replay-value oracle	0.097	11.258	9.601	3.053

Table 4: Repeated DB-level split summary on the expanded 21-token bank. Means and standard deviations are computed over 5 holdout splits. Candidate-level replay value remains close to the oracle boundary across splits; the scene-token student does not.

Selector	Replay fail	Progress (m)	ADE3 (m)	Entropy
proxy top-1	0.602 ± 0.130	25.792 ± 4.552	3.847 ± 0.563	2.100 ± 0.345
scene-token student	0.590 ± 0.110	25.210 ± 5.348	3.940 ± 0.605	2.161 ± 0.294
replay-value, safety-heavy	0.218 ± 0.177	15.856 ± 4.442	7.052 ± 1.112	3.248 ± 0.514
replay-value, balanced	0.289 ± 0.164	19.512 ± 4.513	5.790 ± 1.357	3.185 ± 0.421
replay-value oracle	0.208 ± 0.175	12.197 ± 2.352	8.360 ± 0.982	3.302 ± 0.398

at 19.512 ± 4.513 m, stays within the same oracle-gap tolerance on 3/5 splits, and improves progress over the safety-heavy point on all 5 splits by 3.655 ± 1.719 m.

4.6 Curriculum Targets

Table 5 summarizes the emitted curriculum for the original G_0 decomposition and the expanded-bank follow-up. In the original 9-token study, phase 1 contains 346 safety-recovery targets with high mean regret (226.103), dominated by CRAWL, SLOW_YIELD, EVASIVE_RIGHT, and STOP teachers. After that safety branch is mostly closed, phase 2 contains 228 utility-recovery targets with much smaller mean regret (5.696), dominated by MAINTAIN and NUDGE_RIGHT. Phase 3 contains 281 candidate-expansion requests: these scenes have no safe in-bank teacher and should not be used as ordinary distillation labels. In the expanded-bank follow-up, the same logic yields 223 generation-gap scenes and 453 recoverable safety-regret scenes, showing that candidate expansion shifts mass from generation failure into learnable in-bank regret.

4.7 Interpretation

The result does not say that one fixed risk penalty or one reranker solves driving. It says the lifecycle discovers two different optimization branches and one architectural constraint. The first branch is candidate expansion: richer banks can reduce oracle@K failure from 0.281 to 0.223, so generation support matters. The second branch is recoverable safety: once richer candidates exist, a candidate-level replay-value student can exploit them and stay close to the oracle boundary across repeated DB-level splits. The constraint is that a scene-only token classifier leaves a large gap to that boundary or improves only marginally, so the learning problem must be formulated at candidate level rather than only at scene level.

This is also why the method should be evaluated by lifecycle changes rather than a single top-1 score. A static reranker can reduce top-1 failure, but it does not answer whether the generator's future candidate distribution improves or whether the remaining failures are unrecoverable with the current bank. REGRETBOOT makes that accounting explicit.

Table 5: Bootstrapped curriculum produced by the $G_0 \rightarrow G_1$ lifecycle. Safety and utility phases have in-bank teacher trajectories. Candidate-expansion scenes do not.

Phase	Count	Mean regret	Mean weight	Dominant teachers or requests
safety recovery	346	226.103	2.806	CRAWL: 92, SLOW_YIELD: 96, EVASIVE_RIGHT: 64, STOP: 46, NUDGE_RIGHT: 28
utility recovery	228	5.696	0.767	MAINTAIN: 135, NUDGE_RIGHT: 47, EVASIVE_RIGHT: 17, LANE_RECOVER: 15
candidate expansion	281	n/a	n/a	no in-bank replay-safe teacher; request new samples or richer generator support

5 Discussion

What is new relative to reranking. A learned value calibrator over candidates is not enough by itself; it is close to standard reranking. The contribution here is the training loop around the calibrator: measure oracle@K regret, separate safety from utility from generation gap, distill only recoverable in-bank teachers, and route oracle-failed scenes to candidate expansion. The new empirical point in this version is that the student objective must also match the structure of the interface. Scene-level token classification is too weak once the bank grows. Candidate-level replay value is the effective learning unit.

Why the balanced point matters. The strongest replay-value student is safety-heavy and nearly matches the oracle boundary, but it also sacrifices progress. A balanced replay-value objective, $-250\,\mathbf{1}[\text{fail}] + 2\,\text{progress} - 2\,\text{ADE}_{3s}$, keeps the same 0.097 replay-failure rate as the safety-heavy point on the illustrative split while improving mean progress from 12.276 m to 14.278 m. On repeated DB-level splits it trades some boundary tightness for utility, averaging 0.289 ± 0.164 replay failure versus 0.218 ± 0.177 for the safety-heavy point, but it improves progress over the safety-heavy point in 5/5 splits by 3.655 ± 1.719 m. This is the useful border result in the current draft: candidate-level value learning does not only clone the safest oracle behavior; it can preserve some utility while staying near the oracle boundary on most splits.

Relation to VLA planners. The candidate interface is intentionally model-agnostic. A VLA generator can be plugged in if it emits K candidate trajectories and top-1 metadata. The current repository includes conversion and validation tools for this path, but this paper does not claim VLA top-1 improvement until replay and utility metrics are attached to every generated candidate. The ManeuverToken experiment is therefore a controlled proof of the regret-bootstrap mechanism, not a claim that a released VLA planner has already been improved.

6 Limitations

Log replay is not closed-loop safety. Logged actors do not react to the ego candidate. Replay infeasibility is a public, reproducible future-geometry signal, but it is not equivalent to closed-loop collision under reactive traffic. The correct claim is that replay risk is a useful calibration and bootstrapping signal for candidate planners.

Current generator is a surrogate. The reported lifecycle uses a nine-token ManeuverToken generator. This makes the candidate bank auditable, but it also fixes the candidate distribution. The unchanged 281-scene generation gap is partly a property of that limited bank. A VLA or sampling generator must be evaluated by whether it reduces this gap while preserving candidate diversity.

Current evidence stops at candidate-level selection. The current replay-value student proves that candidate-level learning can exploit the expanded bank, but it does not yet show that a regenerated top-1 policy emits better candidates by itself. A full generator-training paper should run multiple regeneration cycles, report diversity and oracle@K changes, and compare candidate-level reranking against actual generator improvement.

Value weights define a teacher. Equation 1 uses a safety-dominant replay value. Changing the failure penalty or progress/ADE weights changes which utility targets are emitted. This is why the paper reports the lifecycle and curriculum rather than presenting one weight setting as a universal driving objective.

7 Conclusion

Candidate planners need more than a better top-1 scorer. They need to know whether a failure was recoverable inside the candidate bank. REGRETBOOT provides that accounting. On 1,000 public nuPlan interaction scenes, the initial nine-token ManeuverToken generator has a large recoverable safety branch: 346 scenes where a safe in-bank teacher exists but top-1 fails. Expanding the bank to 21 candidates reduces the oracle failure boundary from 0.281 to 0.223, proving that candidate generation matters. On that richer bank, a scene-token student still underperforms badly: on repeated DB-level splits it averages 0.590 ± 0.110 replay failure versus the proxy's 0.602 ± 0.130. In contrast, a candidate-level replay-value student stays close to the oracle boundary: the safety-heavy point averages 0.218 ± 0.177 replay failure versus the oracle's 0.208 ± 0.175 and remains within a 0.05 oracle-gap tolerance on 5/5 splits, while a balanced replay-value objective improves progress over the safety-heavy point on all 5 splits.

The scientific claim is therefore not that replay safety solves closed-loop AV planning. It is that recoverable-regret bootstrapping turns candidate-planner failures into different training tasks: expand the generator when oracle@K still fails, and learn candidate-level replay value when the bank already contains a better action. That is the mechanism a VLA candidate generator should optimize next.

Acknowledgments

Developed independently as a research prototype using public nuPlan-style replay artifacts and internal repository tooling.

References

[1] H. Caesar, J. Kabzan, K. S. Tan, W. K. Fong, E. Wolff, A. Lang, L. Fletcher, O. Beijbom, and S. Omari. nuPlan: A closed-loop ml-based planning benchmark for autonomous vehicles. In *CVPR Workshop on Autonomous Driving*, 2021.

[2] D. A. Pomerleau. ALVINN: An autonomous land vehicle in a neural network. In *Advances in Neural Information Processing Systems*, volume 1, 1989.

[3] S. Ross, G. Gordon, and D. Bagnell. A reduction of imitation learning and structured prediction to no-regret online learning. In *International Conference on Artificial Intelligence and Statistics*, pages 627–635, 2011.

[4] M. Bansal, A. Krizhevsky, and A. Ogale. ChauffeurNet: Learning to drive by imitating the best and synthesizing the worst. In *Robotics: Science and Systems*, 2019.

[5] Y. Hu, J. Yang, L. Chen, K. Li, C. Sima, X. Zhu, S. Chai, S. Du, T. Lin, W. Wang, et al. Planning-oriented autonomous driving. In *IEEE Conference on Computer Vision and Pattern Recognition*, pages 17853–17862, 2023.

[6] K. Chitta, A. Prakash, B. Jaeger, Z. Yu, K. Renz, and A. Geiger. TransFuser: Imitation with transformer-based sensor fusion for autonomous driving. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2022.

[7] A. Brohan, N. Brown, J. Carbajal, Y. Chebotar, X. Chen, K. Choromanski, T. Ding, D. Driess, A. Dubey, C. Finn, et al. RT-2: Vision-language-action models transfer web knowledge to robotic control. *arXiv preprint arXiv:2307.15818*, 2023.

[8] Y. Luo, F. Li, S. Xu, Y. Ji, Z. Zhang, B. Wang, Y. Shen, J. Cui, L. Chen, G. Chen, H. Ye, Z.-X. Yang, and F. Wen. LaST-VLA: Thinking in latent spatio-temporal space for vision-language-action in autonomous driving. *arXiv preprint arXiv:2603.01928*, 2026.